

Segurança de Código na Era da IA

7 Princípios para sair do “Vibe Coding” e desenvolver sistemas à prova de falhas.



Tópicos Abordados



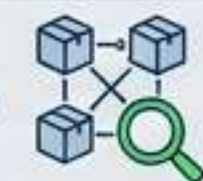
1. Validação e Sanitização de Inputs



2. Autenticação e Autorização (Multitenancy)



3. Gerenciamento de Segredos



4. Gestão de Dependências Externas



5. Rate Limiting (Mitigação de Ataques)



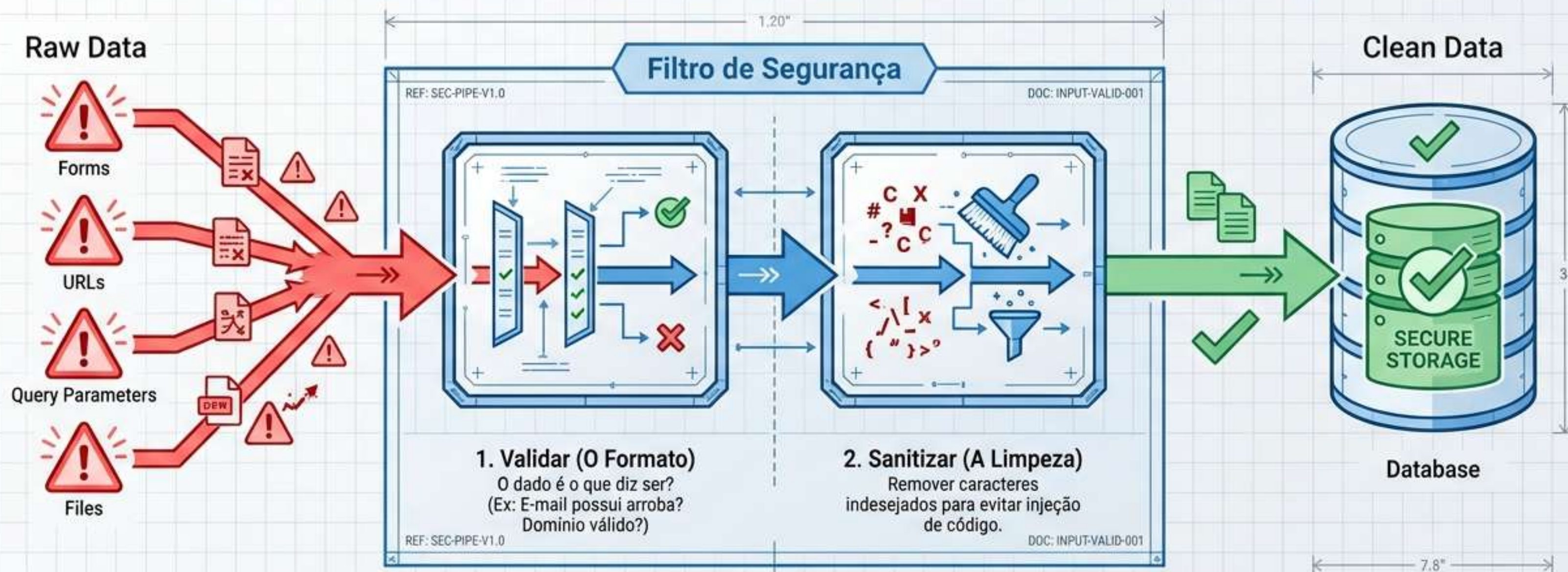
6. Observabilidade e Logs Seguros



7. Princípio do Menor Privilégio

1. O Princípio da Desconfiança: Inputs do Usuário

A Regra de Ouro: Tudo o que vem de fora é inerentemente inseguro. Nunca confie no input.



Red = Raw/Unsafe Data & Vulnerabilities | Blue = Security Processing & Architecture | Green = Verified/Safe Data & Success

1. Na Prática: Validando e Sanitizando Dados

❌ Vibe Coding / Amador

```
// Aceita qualquer coisa e manda pro banco
app.post('/user', (req, res) => {
  db.query(`INSERT INTO users (nome) VALUES ('${req.body.nome})`);

  // Vulnerável a SQL Injection!
});
```



✅ Engenharia Segura / Profissional

```
const validator = require('validator');

app.post('/user', (req, res) => {
  let nome = req.body.nome;

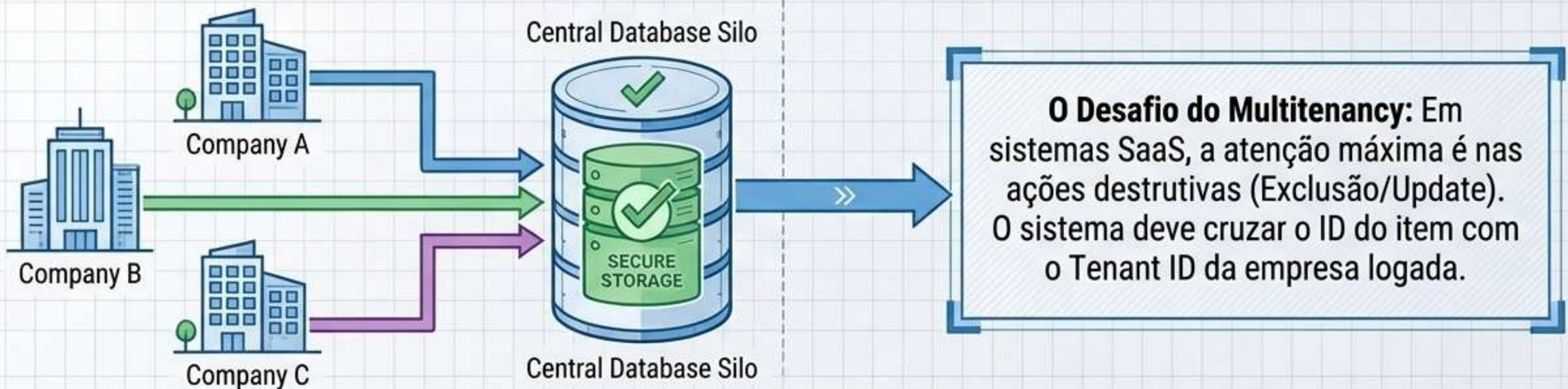
  // 1. Validação (Existe e tem tamanho mínimo?)
  if (!nome || typeof nome !== 'string' || nome.trim().length < 2) {
    return res.status(400).send("Nome inválido.");
  }

  // 2. Sanitização (Remove caracteres perigosos)
  nome = validator.escape(nome.trim());

  // Uso de Prepared Statements
  db.query(`INSERT INTO users (nome) VALUES (?)`, [nome]);
});
```



2. Autenticação, Autorização e Multitenancy



2. Na Prática: Senhas Fortes e Isolamento de Dados

```
const bcrypt = require('bcrypt');  
// 🛡️ 1. Hash de Senha (Autenticação)  
async function criarUsuario(senhaPlana) {  
  const saltRounds = 10;  
  // Transforma a senha em um hash irreversível (Nunca use MD5)  
  return await bcrypt.hash(senhaPlana, saltRounds);  
}
```

Criptografia avançada
isola a identidade.

```
// 🏢 2. Validação Multitenancy (Autorização)  
app.delete('/produtos/:id', (req, res) => {  
  const produtoId = req.params.id;  
  const empresaLogadaId = req.usuario.tenant_id;  
  
  // Exige match do ID do produto E da empresa logada  
  db.query(  
    `DELETE FROM produtos WHERE id = ? AND tenant_id = ?`,  
    [produtoId, empresaLogadaId]  
  );  
});
```

Chave do Multitenancy:
Garante que a empresa A
não apague dados da
empresa B.

3. Gerenciamento de Segredos (Chaves e Senhas)

O Erro Fatal (Don'ts)



O Erro Fatal: Commitar chaves de API, senhas ou tokens em repositórios (mesmo os privados).

A Solução (Do's)



- **A Solução:** Isolar variáveis de ambiente (arquivos `.env` locais).
- **O Escudo (`.gitignore`):** Arquivos com senhas reais nunca saem da sua máquina.
- **O Modelo (`.env.example`):** Um mapa para a equipe com os nomes das variáveis necessárias, mas sem valores.



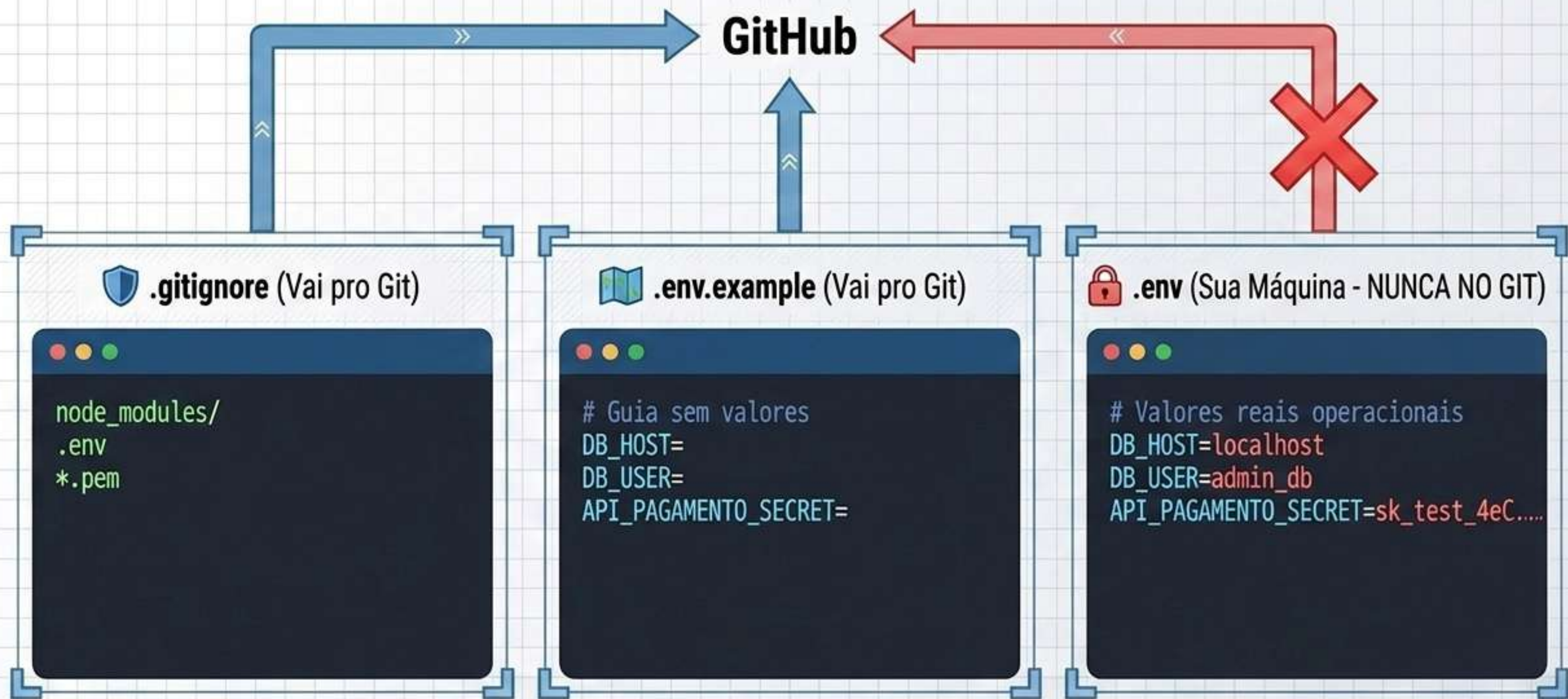
PRO-TIP: Vazou um segredo no Git? Fazer um novo commit apagando não adianta, o histórico guarda tudo.

Solução: Delete o repositório e recomece (ou use cofres corporativos como o Infisical).

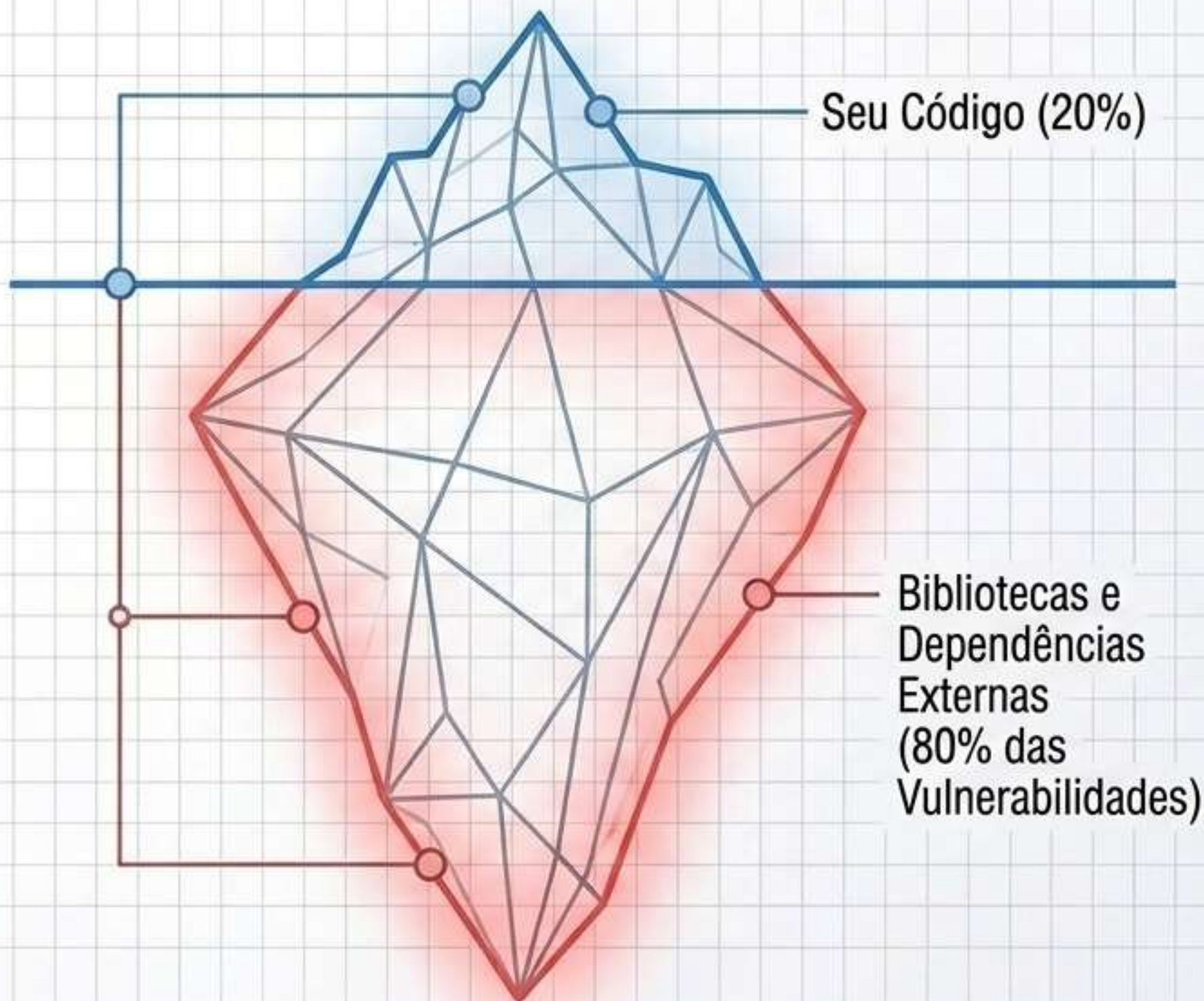


INFISICAL

3. Na Prática: A Tríade da Configuração Segura



4. Dependências: O Perigo Invisível do Código de Terceiros



- **O Paradoxo:** Instalar pacotes por conveniência importa os problemas de segurança de outros desenvolvedores.
- **Efeito Cascata:** Uma biblioteca possui suas próprias dependências, criando uma extensa árvore de riscos.
- **CrITÉrios de Escolha:**
 1. Projetos validados pelo mercado com manutenção ativa.
 2. Bibliotecas com o mínimo possível de sub-dependências.

4. Na Prática: Automatizando a Auditoria de Pacotes

 Auditoria via CLI

```
$ npm audit

> found 3 vulnerabilities (1 low, 2 high) in 1523 scanned packages
> run `npm audit fix` to fix them, or `npm audit fix --force` for breaking changes
```

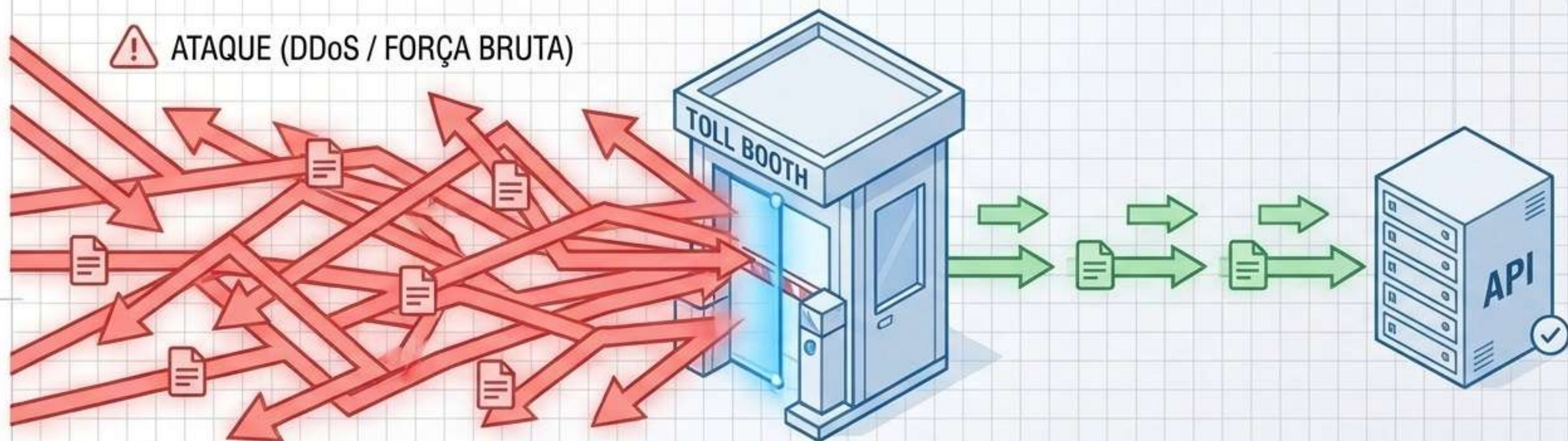
 Monitoramento Contínuo (GitHub Dependabot .github/dependabot.yml)

```
version: 2
updates:
  - package-ecosystem: "npm"
    directory: "/"
    schedule:
      interval: "weekly" # Procura brechas toda semana
```



Ferramentas avisam ativamente se aquele pacote 'mágico' que você instalou foi recém-descoberto como uma ameaça.

5. Rate Limiting: Prevenção Contra Sobrecarga



- O Risco (Força Bruta / DDoS):

APIs desprotegidas podem receber milhões de requisições por segundo até o servidor cair.

- O Mecanismo:

Bloquear excesso de requisições baseando-se em uma janela de tempo.

- Parâmetros de Bloqueio:

Por Endereço IP, Por API Key, ou Por Usuário logado.

- Tecnologias:

API Gateways, Cloudflare, ou bancos em memória ultra-rápidos (Redis).

5. Na Prática: Implementando o Toll Booth na API

```
const express = require('express');
const rateLimit = require('express-rate-limit');
const app = express();

// 🛡️ Criando a regra de limitação
const limitador = rateLimit({
  → windowMs: 1 * 60 * 1000, // Janela de tempo: 1 minuto
  → max: 60, // Limite máximo: 60 requisições por IP
  message: "Muitas requisições deste IP. Tente novamente em 1 minuto."
});

// Aplica a regra para proteger todas as rotas
app.use(limitador);

app.post('/login', (req, res) => {
  // Se um robô tentar adivinhar a senha 61 vezes no mesmo minuto,
  // a requisição é bloqueada antes de chegar nesta linha.
  res.send('Login processado');
});
```

Garante a sobrevivência do servidor sob ataque.

6. Observabilidade e a Armadilha dos Logs



- **A Necessidade:** Logs são vitais para rastrear erros e entender o comportamento do sistema em produção.
- **O Risco de Vazamento:** A preguiça de dar `console.log(objetoInteiro)` acaba imprimindo dados críticos em arquivos de texto plano.
- **A Regra Absoluta:** Nunca envie informações sensíveis (PII) para arquivos de log.
- **Exemplos Críticos a Evitar:** Senhas de usuários, CPFs e prompts inteiros de IA contendo informações confidenciais.

6. Na Prática: Mascarando Dados em Produção

❌ LOG PERIGOSO: Vaza texto puro

```
console.log(`Tentativa de login - IP: ${req.ip} | Senha: ${req.body.senha}`);  
// Saída: Tentativa de login - IP: 192.168.1.5 | Senha: admin123
```

✅ LOG SEGURO: Mascarando (Redacting) dados sensíveis

```
function mascararSenha(senha) {  
  return senha ? '*'.repeat(senha.length) : 'N/A';  
}  
const tentativaSenha = mascararSenha(req.body.senha);  
  
// O sistema registra o evento, mas mantém o dado seguro  
console.log(`[INFO] Login | IP: ${req.ip} | Senha: ${tentativaSenha}`);  
// Saída: [INFO] Login | IP: 192.168.1.5 | Senha: *****
```

7. O Princípio do Menor Privilégio (PoLP)

⚠ O Paradoxo da Preguiça:
Dar "acesso total" para facilitar o desenvolvimento cria brechas massivas.



✓ A Filosofia:
Qualquer recurso (Usuário de BD, IAM AWS, CORS) deve ter apenas as permissões estritamente necessárias.

A Última Linha de Defesa: Se um invasor contornar as validações e injetar um comando DROP TABLE, ele falhará se o usuário do banco só possuir privilégio de SELECT.

7. Na Prática: A Última Linha de Defesa

Exemplo 1: Camada de Dados

❌ LOG PERIGOSO: Vaza texto puro

```
// ❌ ERRO COMUM: Poder absoluto  
GRANT ALL PRIVILEGES ON loja_db.* TO 'api_user'@'localhost';
```

✅ LOG SEGURO: Apenas permissões vitais

```
// ✅ SEGURO: Apenas permissões vitais  
GRANT SELECT ON loja_db.produtos TO 'api_user'@'localhost';  
GRANT INSERT, UPDATE ON loja_db.vendas TO 'api_user'@'localhost';
```

Exemplo 2: Camada de Rede

❌ LOG PERIGOSO: Liberado para qualquer origem

```
// ❌ LIBERADO PARA QUALQUER ORIGEM  
app.use(cors());
```

✅ LOG SEGURO: Restrito à origem oficial

```
// ✅ RESTRITO: Apenas seu site front-end oficial  
app.use(cors({ origin: 'https://meu-site-oficial.com' }));
```

**“Codifique como um programador.
Faça o deploy como um paranoico.”**